

DSAA3073: Theories in Data Science

Tutorial 1B: From Shape to Feasible Learning

150-minute core tutorial · 15-minute break · 60-minute protected buffer

Wei WANG · HKUST(GZ) · 2026-08-28

Explorative projection

LESSON BACKBONE

Even when a data claim is statistically meaningful, what makes its objective optimizable and its solution computationally attainable?

Tutorial 1A gave us three items that we may now use productively: Hoeffding's inequality for bounded independent averages, the idea that an L_p norm selects a geometry, and the notation $\|x\|_p$. We will keep those meanings fixed while asking three different questions about one learning task: Is the feasible region and objective convex? What can a gradient certify and change? Can the required computation finish in polynomial time?

The core path has three blocks: **shape before algorithm** (45 minutes), **first-order reasoning and gradient updates** (60 minutes), and **statistical, optimization, and computational feasibility** (45 minutes). Take the scheduled 15-minute break after the first block. The final 60 minutes are a protected recovery period and introduce no required concept.

Clock	Mathematical work	Status
0–45	Convex sets, convex functions, and decisive witnesses	45 min core
45–60	Scheduled break	15 min
60–120	First-order lower bounds and gradient-method traces	60 min core
120–165	Encoded size, polynomial time, and three-axis diagnosis	45 min core
165–225	Questions, slower derivations, alternative cases	60 min buffer

Shape before algorithm

Suppose a model is represented by $w \in \mathbb{R}^d$. A constraint such as $\|w\|_2 \leq 1$ says which parameter vectors are feasible, while an objective $f(w)$ tells us which feasible vector is preferred. These are separate objects: the set can be convex while the objective is not, or conversely.

The useful experiment is interpolation: choose two objects and inspect $\lambda x + (1 - \lambda)y$ for $0 \leq \lambda \leq 1$. A few attractive points cannot prove a statement about every pair, but one escaping interpolation can refute it. We therefore need criteria that say exactly what stays inside a feasible set and what stays below a chord.

Convex sets keep every segment

Definition: Convex set

A set $C \subseteq \mathbb{R}^d$ is **convex** if, for every $x, y \in C$ and every $\lambda \in [0, 1]$,

$$\lambda x + (1 - \lambda)y \in C. \quad (1)$$

The point $\lambda x + (1 - \lambda)y$ is a convex combination of x and y .

The quantifiers determine the proof strategy. To establish convexity, we must handle arbitrary x , y , and λ . To refute convexity, one counterexample is enough: find two points in the set and one point on their segment that leaves the set.

Worked example: An L2 ball is convex

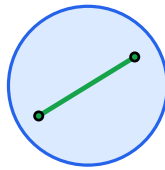
Let $C = \{w : \|w\|_2 \leq 1\}$, choose $x, y \in C$, and let $\lambda \in [0, 1]$. The triangle inequality and homogeneity give

$$\|\lambda x + (1 - \lambda)y\|_2 \leq \lambda\|x\|_2 + (1 - \lambda)\|y\|_2 \leq 1. \quad (2)$$

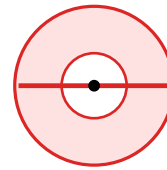
Therefore the entire segment stays in C . The same proof works for every norm ball $\{w : \|w\|_p \leq r\}$ with $p \geq 1$ and $r \geq 0$.

An annulus behaves differently. Let $A = \{w \in \mathbb{R}^2 : 1 \leq \|w\|_2 \leq 2\}$. Both $x = (1, 0)$ and $y = (-1, 0)$ lie in A , but their midpoint is $(0, 0)$, which does not. This single midpoint is a complete nonconvexity witness.

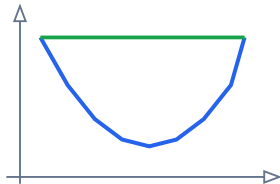
Set: segment stays inside



Set: one segment escapes



Function: graph below chord



Function: violated chord

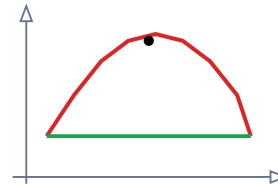


Figure 1: The same interpolation parameter governs all four panels. One escaping segment or one point above its chord refutes convexity.

IN-CLASS CHECK

Prompt: In the upper-right panel, mark two feasible endpoints and their escaping midpoint. In the lower-right panel, mark the two chord endpoints and the graph point above the chord. Write one inequality for each failure.

Hint: Use $\lambda = \frac{1}{2}$ for the pictured witnesses. A set failure is a non-membership statement; a function failure reverses the required chord inequality.

Answer:

For the annulus, $x = (1, 0)$ and $y = (-1, 0)$ are feasible but $\frac{1}{2}x + \frac{1}{2}y = (0, 0) \notin A$. For the pictured function, choose the two red endpoints a, b and their midpoint $m = \frac{1}{2}a + \frac{1}{2}b$. The black point records $f(m) < (\frac{1}{2}f(a) + \frac{1}{2}f(b))$, which is exactly a violated midpoint chord.

Convex functions lie below their chords

Definition: Convex function

Let C be a convex set. A function $f : C \rightarrow \mathbb{R}$ is **convex** if, for every $x, y \in C$ and $\lambda \in [0, 1]$,

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y). \quad (3)$$

The left side evaluates the function at an interpolated input. The right side is the height of the chord joining $(x, f(x))$ and $(y, f(y))$.

The phrase “on a convex domain” is part of the definition. A convex feasible set says interpolated **parameters** remain legal. A convex objective says the objective at those interpolated parameters lies no higher than the interpolated endpoint values. Neither statement silently supplies the other.

Worked example: A quadratic passes the chord test

For $f(w) = w^2$ on $\mathbb{R}$, expand the difference between the chord height and the function value:

$$\lambda x^2 + (1 - \lambda)y^2 - (\lambda x + (1 - \lambda)y)^2 = \lambda(1 - \lambda)(x - y)^2 \geq 0. \quad (4)$$

Hence $f(\lambda x + (1 - \lambda)y)$ is no greater than the chord height, so f is convex.

IN-CLASS CHECK

Prompt: For each object, either justify convexity or give a specific witness against it: (i) $\{w : \|w\|_1 \leq 2\}$; (ii) $\{w \in \mathbb{R}^2 : \|w\|_2 = 1\}$; (iii) $f(w) = (w - 2)^2$; (iv) $g(w) = (w^2 - 1)^2$.

Hint: For a set, test a segment. For a one-dimensional function, a violated midpoint chord is enough to refute convexity.

Answer:

(i) is convex by the norm-ball argument. (ii) is not: (1, 0) and (0, -1) are on the unit circle, but their midpoint is not. (iii) is convex because translating a quadratic does not change the nonnegative chord difference. (iv) is not convex: $g(1) = g(-1) = 0$, while $g(0) = 1$ lies above the midpoint chord height.

Key Takeaway

Convexity is a quantified interpolation statement. Proving it needs an arbitrary-pair argument; disproving it needs one decisive segment or chord. Keep the feasible-set condition separate from the objective condition.

What a gradient can certify and change

Convexity changes the meaning of local derivative information. For an arbitrary differentiable function, a tangent describes only nearby behavior. For a convex function, that tangent is a lower bound everywhere on the domain.

The chord inequality is now available, and derivatives are part of the course prerequisites. The next question is whether that local derivative can certify anything globally. Only after answering it will we use the derivative to build an update: a direction alone does not yet specify a step size or a new iterate.

From a chord to a tangent lower bound

Theorem: First-order condition for differentiable convex functions

Let $f : C \rightarrow \mathbb{R}$ be differentiable on an open convex set C . Then f is convex if and only if, for all $x, y \in C$,

$$f(y) \geq f(x) + \nabla f(x)^T(y - x). \quad (5)$$

Thus the tangent hyperplane at x is a global lower bound on f .

Here is the forward direction, which is the direction we will use. Fix x, y and set $z_t = x + t(y - x)$ for $0 < t \leq 1$. Convexity gives

$$f(z_t) \leq (1 - t)f(x) + tf(y). \quad (6)$$

Rearrange before taking a limit:

$$\frac{f(x + t(y - x)) - f(x)}{t} \leq f(y) - f(x). \quad (7)$$

As $t \rightarrow 0^+$, the left side becomes the **directional derivative**: the derivative at zero of the one-variable function $t \mapsto f(x + t(y - x))$. Its value is $\nabla f(x)^T(y - x)$. Therefore $f(y) - f(x) \geq \nabla f(x)^T(y - x)$. Notice the order of ideas: convexity makes the chord inequality available; differentiability turns its short-segment limit into a tangent.

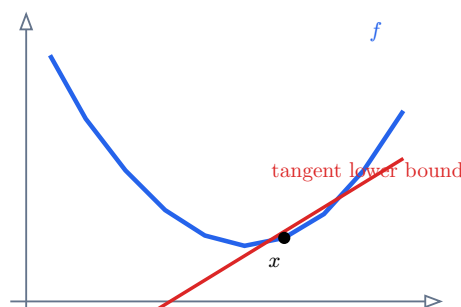


Figure 2: At the contact point the tangent and the function agree; away from it, convexity keeps the function above the tangent.

Worked example: The tangent certificate for a quadratic

Let $f(w) = \frac{1}{2}(w - 3)^2$, so $f'(w) = w - 3$. At $x = 0$ the tangent lower bound is

$$f(y) \geq f(0) + f'(0)(y - 0) = \frac{9}{2} - 3y. \quad (8)$$

Direct subtraction confirms the certificate:

$$f(y) - \left(\frac{9}{2} - 3y\right) = \frac{1}{2}y^2 \geq 0. \quad (9)$$

At $x = 0$ the derivative is negative, but the lower-bound statement concerns every y ; it is stronger than the informal phrase “the graph slopes down.”

IN-CLASS CHECK

Prompt: Suppose a differentiable convex f has $\nabla f(w^*) = 0$. Use one line to show that w^* is a global minimizer.

Hint: Insert $x = w^*$ into the first-order condition.

Answer:

For every y , $f(y) \geq f(w^*) + 0^T(y - w^*) = f(w^*)$. Therefore no point has a smaller value. This conclusion uses convexity; a zero gradient alone is not enough for a nonconvex function.

The unconstrained gradient method**Definition: Gradient method (gradient descent)**

For a differentiable objective $f : \mathbb{R}^d \rightarrow \mathbb{R}$, an initial point w_0 , and a positive step size η , the unconstrained gradient method updates

$$w_{t+1} = w_t - \eta \nabla f(w_t). \quad (10)$$

When this update is used to minimize an objective, the standard textbook name is **gradient descent**. The gradient $\nabla f(w_t)$ is the vector of partial derivatives at the current iterate. The negative sign moves against the local direction of greatest increase. This Tutorial does not project the update onto a constraint set.

The rule is a state transition, not a convergence theorem. To execute it we must know the current iterate, evaluate the current gradient, declare η , and then compute the next iterate. Whether values decrease depends on the objective and on the step size.

Worked example: Two updates, with every state visible

Continue with $f(w) = \frac{1}{2}(w - 3)^2$, start at $w_0 = 0$, and take $\eta = \frac{1}{2}$. Since $f'(w) = w - 3$,

$$f'(w_0) = -3, \quad w_1 = 0 - \left(\frac{1}{2}\right)(-3) = \frac{3}{2}, \quad (11)$$

$$f'(w_1) = -\frac{3}{2}, \quad w_2 = \frac{3}{2} - \left(\frac{1}{2}\right)\left(-\frac{3}{2}\right) = \frac{9}{4}. \quad (12)$$

The objective values are $f(w_0) = \frac{9}{2}$, $f(w_1) = \frac{9}{8}$, and $f(w_2) = \frac{9}{32}$. Each step halves the distance to the minimizer $w^* = 3$ and quarters the objective gap.

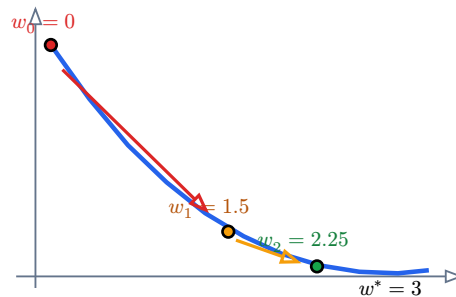


Figure 3: For $\eta = \frac{1}{2}$, the three iterates follow one fixed quadratic. Arrow lengths show iterate displacement, not gradient magnitude.

Why the step size matters

For this quadratic the error $e_t = w_t - 3$ obeys an exact recurrence:

$$e_{t+1} = w_t - \eta(w_t - 3) - 3 = (1 - \eta)e_t. \quad (13)$$

This makes four behaviors transparent.

η	Error multiplier	Behavior for this quadratic
$\frac{1}{2}$	$\frac{1}{2}$	same side, distance halves
$\frac{3}{2}$	$-\frac{1}{2}$	crosses the minimizer, distance halves
2	-1	alternates forever; objective does not decrease
$\frac{5}{2}$	$-\frac{3}{2}$	alternates with growing distance; diverges

IN-CLASS CHECK

Prompt: Start again from $w_0 = 0$ for $f(w) = \frac{1}{2}(w - 3)^2$. Predict and then compute w_1 and w_2 for $\eta = \frac{3}{2}$ and $\eta = \frac{5}{2}$. Which case overshoots but converges, and which overshoots and diverges?

Hint: Use $w_{t+1} = w_t - \eta(w_t - 3)$ or the error multiplier $1 - \eta$.

Answer:

With $\eta = \frac{3}{2}$, $w_1 = \frac{3}{2}$ and $w_2 = \frac{9}{4}$; the errors are $\frac{3}{2}$ and $-\frac{3}{2}$ in alternating signs, so the method overshoots but converges. With $\eta = \frac{5}{2}$, $w_1 = \frac{15}{2}$ and $w_2 = \frac{45}{2}$; error magnitudes grow from 3 to $\frac{9}{2}$ to $\frac{45}{2}$, so the method diverges. “Overshoot” by itself is not the diagnosis; the magnitude of $1 - \eta$ is.

Key Takeaway

Convexity turns the tangent into a global lower bound. The gradient method then turns the current gradient into an explicit state change, but a useful update still requires a declared and suitable step size. A valid formula is not automatically a convergence guarantee.

Three ways a learning claim can fail

We can now estimate a fixed predictor's mean loss and optimize a simple convex objective. A third question remains: how does the work scale with the size of the encoded input? Statistical sufficiency, favorable geometry, and efficient computation are different guarantees.

The same task may contain bounded losses, a parameter geometry, an objective, and an implementation. If the full claim fails, we should locate the broken link before reaching for a theorem: a probability bound does not count operations, and a convexity proof does not specify code. We will audit the statistical, optimization, and computational claims separately and combine only the conclusions that have evidence.

Polynomial in which input size?

Definition: Polynomial-time algorithm

After fixing a computation model and an encoding, an algorithm is **polynomial time** if its number of elementary operations is at most cN^k for constants c, k independent of the instance, where N is the number of symbols or bits in the encoded input. The relevant input parameters must be stated; hiding one of them can change the classification.

The encoding clause prevents a common mistake. Suppose an integer M is written in binary, so its representation has about $\log_2 M$ bits. A loop that runs M times is polynomial in the **numeric value** M , but exponential in its encoded length $N \approx \log_2 M$, because $M \approx 2^N$.

Worked example: Linear scan versus exhaustive enumeration

Reading N stored loss values once takes $O(N)$ operations. By contrast, evaluating every binary vector in $\{0, 1\}^d$ takes 2^d evaluations. Even if one evaluation is polynomial in d , the full enumeration is exponential in d . A faster processor changes constants; it does not turn 2^d into a polynomial.

IN-CLASS CHECK

Prompt: An algorithm receives a d -bit description and uses $O(d^3)$ preprocessing followed by $O(2^d d)$ search. Is the complete algorithm polynomial time in d ? What if d is fixed once and never scales?

Hint: Classify the sum by its fastest-growing term, then distinguish a fixed constant from an input parameter.

Answer:

The complete runtime is exponential in variable d because $2^d d$ dominates every polynomial. If d is genuinely fixed outside the problem sequence, then 2^d is a constant and the remaining dependence may be polynomial in other inputs. The runtime claim must say which parameters scale.

Enough data is not the same as a fast search

Definition: Computational-statistical gap

A **computational-statistical gap** occurs when the available information or sample guarantees permit the desired statistical accuracy, while no computationally efficient method is known or available under the same stated assumptions. It is a separation between what the data can identify and what the algorithm can feasibly compute.

Worked example: A qualified positive gap from one-way functions

A **one-way function** is easy to evaluate but, by assumption, no randomized polynomial-time algorithm can invert it with non-negligible success. Chapter 8 of the source considers inverse functions indexed by polynomial-length secret keys. It establishes two facts for this family: polynomially many labeled examples suffice statistically, yet a polynomial-time learner would yield an efficient inverter and contradict the one-way-function assumption.

This is a genuine but conditional computational-statistical gap. The statistical and computational claims concern the same inverse-function family, and the computational obstruction names its assumption. We use the source result as stated; Chapter 1 does not derive its sample theorem or its cryptographic reduction.

Contrast that example with a finite collection of 2^d candidate predictors. Suppose a separate statistical analysis has already supplied a polynomial sample guarantee, while the current exact implementation evaluates all 2^d candidates. This proves that **enumeration** is inefficient; it does not prove that the learning task is inherently hard. The class may have structure that supports a faster algorithm. A slow implementation is therefore a warning to look for a gap, not a positive gap example by itself.

Same task	Statistical	Optimization	Computational
Assumptions	independent losses in $[0, 1]$; chosen ε, δ	$C = \{w : \ w\ _2 \leq 1\}$; quadratic objective	encoded dimension d ; exact selector enumerates 2^d states
Guarantee	fixed-predictor mean-loss error is controlled	interpolation is feasible; tangents are global lower bounds	
Evidence status	supported by Hoeffding	supported by segment and chord checks	no polynomial runtime bound for the stated selector
Do not infer	therefore the optimizer is fast	therefore the data are sufficient	therefore the output is statistically accurate

Table 1: Three audits of the same bounded-loss pipeline. Red statements are unsupported cross-column inferences.

IN-CLASS CHECK

Prompt: Fill the missing computational guarantee cell. Then state what must change before this same pipeline can be called polynomial time.

Hint: Classify the declared 2^d selector; do not replace an absent guarantee with a hope about another algorithm.

Answer:

The missing cell is: **no polynomial-time guarantee for the stated exact selector; its runtime is exponential in d** . To obtain a positive polynomial-time claim, the team must give a different implementation and a runtime bound polynomial in all declared encoded inputs. Hoeffding and the convexity of a continuous relaxation do not supply that implementation.

IN-CLASS CHECK

Prompt: Which case is a genuine computational-statistical gap: (i) one attempted selector is exponential but no hardness result is known, or (ii) under the one-way-function assumption, polynomially many examples suffice for the inverse-function family but an efficient learner would break the assumption?

Hint: A positive instance must pair a statistical guarantee and a computational obstruction for the same task under the same conditions.

Answer:

Case (ii) is the qualified positive instance. Case (i) diagnoses only a slow implementation, an untied structured algorithm might still be efficient.

IN-CLASS CHECK

Prompt: Match each situation to the governing foundation and state one unsupported conclusion: (A) losses are bounded but observations may be dependent; (B) the feasible set contains two points but not their midpoint; (C) an exact method runs in 2^d time although the accuracy guarantee needs only polynomially many samples; (D) a convex gradient update uses an undeclared step size.

Hint: Use the words statistical, geometric, optimization, and computational only after naming the missing assumption or certificate.

Answer:

(A) is statistical: independence is missing, so the displayed Hoeffding guarantee is unsupported. (B) is geometric: the midpoint refutes convexity of the feasible set, so convex-optimization conclusions are unsupported. (C) is computational: sample sufficiency does not make enumeration efficient. (D) is optimization-related: the update can be computed symbolically, but decrease or convergence is unsupported until the step size and objective conditions are checked.

Chapter 1 boundary

Foundation	Question it answers	Representative certificate
Concentration	Can a finite average approximate an expectation?	Hoeffding with boundedness and independence
Geometry	What counts as size, interpolation, and a legal parameter?	$\ w\ _p$ and a segment test
Convex analysis	What global statement follows from a local derivative?	$f(y) \geq f(x) + \nabla f(x)^T(y-x)$
Algorithms	Can the required computation scale to encoded inputs?	a polynomial runtime bound
Computational-statistical separation	Can samples suffice while efficient learning remains unavailable?	a same-task positive instance with its hardness assumption

Key Takeaway

A learning guarantee is only as strong as its weakest unproved link. Probability controls sampling uncertainty, convex geometry controls interpolation and first-order reasoning, and complexity analysis controls scalable implementation. Diagnose the missing link before choosing the tool.

Standard language to carry forward

Term	Meaning here	Representative form
Convex set	Every segment between two points remains in the set.	$\lambda x + (1 - \lambda)y \in C$
Convex function	Its value at an interpolated input lies below the endpoint chord.	$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$
First-order condition	For differentiable convex f , every tangent hyperplane is a global lower bound.	$f(y) \geq f(x) + \nabla f(x)^T(y-x)$
Gradient method / gradient descent	Subtract a positive step size times the current gradient to minimize an objective.	$w_{t+1} = w_t - \eta \nabla f(w_t)$
Polynomial time	Runtime is polynomial in declared encoded input sizes under a fixed computation model.	$O(N^k)$
Computational-statistical gap	The data support the target accuracy, but an efficient computation is unavailable under the same assumptions.	information $\neq$ implementation

Optional deep dive

OPTIONAL DEEP DIVE - 0 CLASS MINUTES

Question: What would it take to replace “enumeration is slow” with an NP-hardness claim?

Answer.

An NP-hardness claim is not a synonym for “the algorithm we tried is exponential.” One normally gives a polynomial-time reduction from a problem already known to be NP-hard: transform each instance of the known problem into an instance of the learning or optimization problem so that a polynomial-time solver for the latter would solve the former. The reduction must preserve the relevant yes/no or approximation structure and must itself run in polynomial time.

Such a result is conditional evidence about the absence of a polynomial-time exact method, typically under the conjecture $P \neq NP$. It does not say that every heuristic is useless, that approximation is impossible, or that every fixed-dimensional instance is hard. Reductions and complexity classes are deferred; no Chapter 1 core check depends on them.

Post-class or self-study. Not a prerequisite for the core path.

Source notes

- M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*, 2nd ed., Appendix B.2–B.3, printed pp. 415–421 / PDF pp. 432–438: convex sets, convex functions, first-order characterization, and constrained optimization context.
- S. Shalev-Shwartz and S. Ben-David, *Understanding Machine Learning: From Theory to Algorithms*, Section 12.1.1, printed/PDF pp. 156–160, and Sections 14.1–14.2, printed/PDF pp. 185–191: convexity, tangent lower bounds, and the unconstrained gradient update.
- Shalev-Shwartz and Ben-David, Chapter 8, printed/PDF pp. 101–110: encoded inputs, efficient learning, exhaustive search, and the separation between sample and computational complexity.
- DSAA3073 Week 1 Learning Sheet v4, pp. 23–41, used for course scope, motivating questions, and sequencing. The textbook sources above control the mathematical statements; unsupported or overbroad claims in the historical sheet are not carried forward.